Memory Problems in Sequence Modeling

Bingran Li

The Chinese University of Hong Kong, Shenzhen

bingranli@link.cuhk.edu.cn

November 29, 2025

Research Questions (Added after the group meeting on 2025/11/19)

- What is the overall cost function of the optimization processes in Nested learning (e.g. Nested MLPs)?
- In Nested MLPs, can the MLP with slow frequencies be connected to corresponding blocks? (Go beyond non-hierarchical sturctures that only pile up decoder blocks with the same internal design.)

Contents

Language Modeling

Goal

Learn the joint probability of word sequences:

$$P(w_1, w_2, \ldots, w_T)$$

Language Modeling

Goal

Learn the joint probability of word sequences:

$$P(w_1, w_2, \ldots, w_T)$$

Auto-regressive Decomposition

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_{1:t-1})$$

Language Modeling

Goal

Learn the joint probability of word sequences:

$$P(w_1, w_2, \ldots, w_T)$$

Auto-regressive Decomposition

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_{1:t-1})$$

Neural Language Model

Parameterize with neural network f_θ:

$$P_\theta(w_t \mid w_{1:t-1}) = f_\theta(w_{1:t})$$

Architecture: Transformer, RNN, SSM, Linear Attention, ...

Language Modeling

Training Objective

Minimize Negative Log-Likelihood (NLL):

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P_\theta(w_t \mid w_{1:t-1})$$

Optimizer: SGD, Adam, AdamW, ...

From Words (Text) to Vectors

Tokenization: Given text $w_{1:T}$, convert it into a sequence of discrete **tokens** $\in \{1, 2, \ldots, V\}$, where V is the vocabulary size.

From Words (Text) to Vectors

Tokenization: Given text $w_{1:T}$, convert it into a sequence of discrete **tokens** $\in \{1, 2, \ldots, V\}$, where V is the vocabulary size.

Let's tokenize: "A hippopotamus ate my homework."

Vocab Type	Example	Ex. length
character-level	['A', ' ', 'h', 'i', 'p', 'p', 'o', 'p', 'o', 't', 'a', 'm', 'u', 's', ' ', 'a', 't', 'e', ' ', 'm', 'y', ' ', 'h', 'o', 'm', 'e', 'w', 'o', 'r', 'k', '.']	31
subword-level	['A', 'hip', '##pop', '##ota', '##mus', 'ate', 'my', 'homework', '.']	9
word-level	['A', 'hippopotamus', 'ate', 'my', 'homework']	5

From Words (Text) to Vectors

Tokenization: Given text $w_{1:T}$, convert it into a sequence of discrete **tokens** $\in \{1, 2, \ldots, V\}$, where V is the vocabulary size.

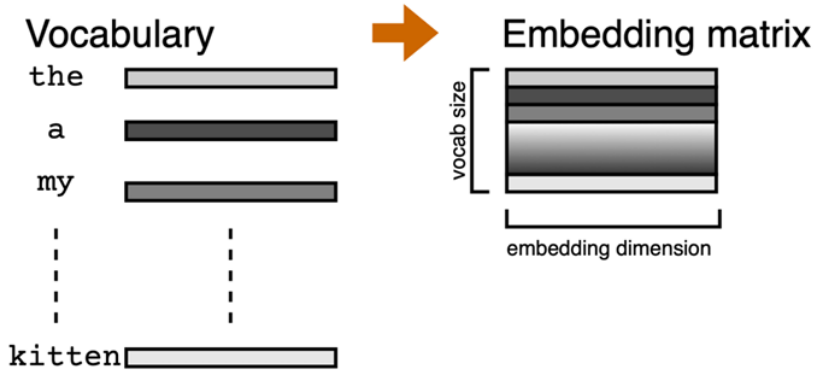
Let's tokenize: "A hippopotamus ate my homework."

Vocab Type	Example	Ex. length
character-level	['A', ' ', 'h', 'i', 'p', 'p', 'o', 'p', 'o', 't', 'a', 'm', 'u', 's', ' ', 'a', 't', 'e', ' ', 'm', 'y', ' ', 'h', 'o', 'm', 'e', 'w', 'o', 'r', 'k', '.']	31
subword-level	['A', 'hip', '##pop', '##ota', '##mus', 'ate', 'my', 'homework', '.']	9
word-level	['A', 'hippopotamus', 'ate', 'my', 'homework']	5

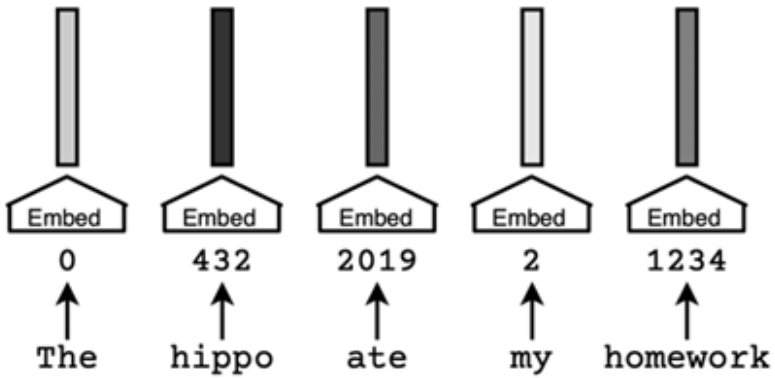
Tokenization are usually different across pruduct-grade models.

From Words (Text) to Vectors

Embedding: make each token correspond to a d_e-dimensional vector.



(a) Embedding matrix



(b) Token embeddings

Summary: text -> tokens -> token embeddings (vectors).

Contents

What is memory in human brain

Fact [Sch12]:

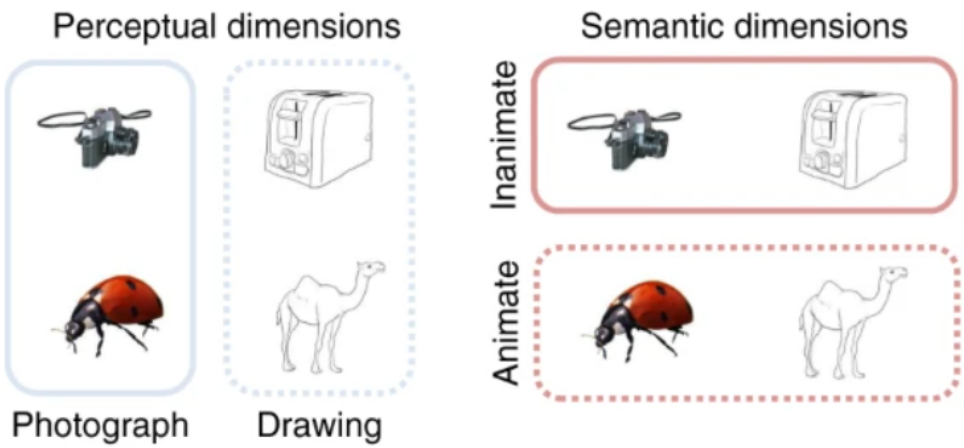
- Human memory is not a literal reproduction of the past, but instead relies on **constructive processes** that are sometimes prone to error and distortion.

What is memory in human brain

Fact [Lin+19]:

- Memory retrieval is a hierarchical, multi-layered process that prioritises semantically meaningful information over perceptual details.

When seeing an object, low-level perceptual details -> high-level conceptual features
Associative memory recall: conceptual features -> perceptual details



Contents

Transformers [Vas+23]

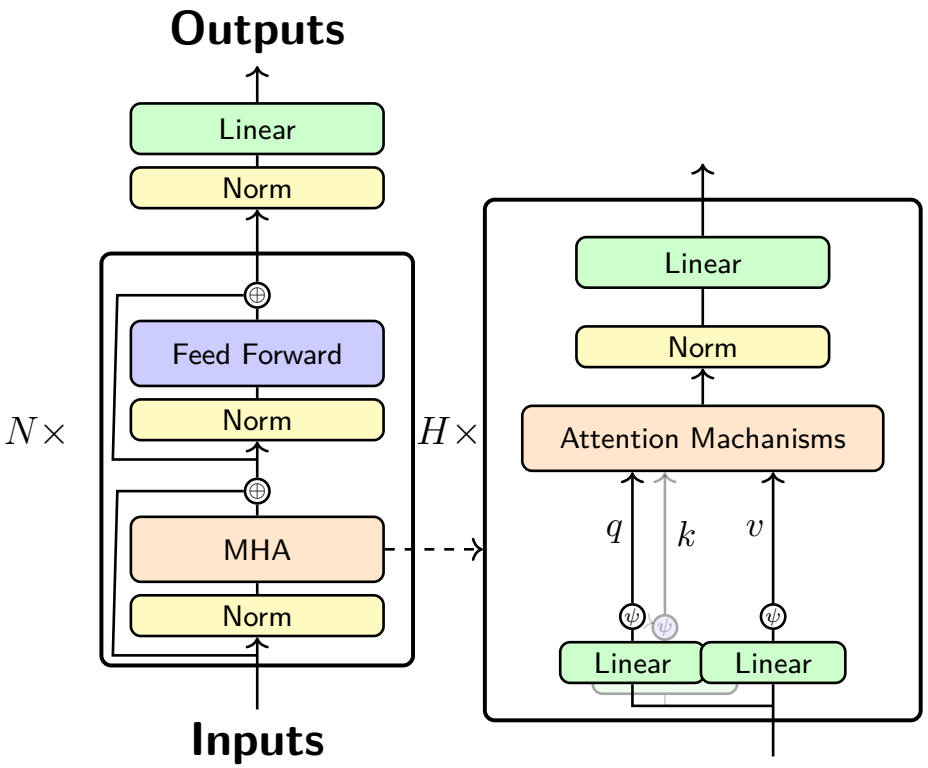


Figure: Auto-regressive Transformer architecture. (LLaMA3-style)

Input: Token embeddings $\{x_t\}_{t=1}^{T}$.

- **Structures of a Decoder Block:** For a token embedding $x_t \in \mathbb{R}^d$,
  $x_t = x_t + \text{MHA}(\text{Norm}(x_t))$,
  $x_t = x_t + \text{FeedForward}(\text{Norm}(x_t))$
- Each module in a decoder block has many variants. e.g. FeedForward: SwiGLU, Norm: LayerNorm, RMSNorm.
- Today we focus on variants of **Attention Mechanisms**: Linear Attention, Gated Delta Net, Kimi Linear.

Transformers [Vas+23]

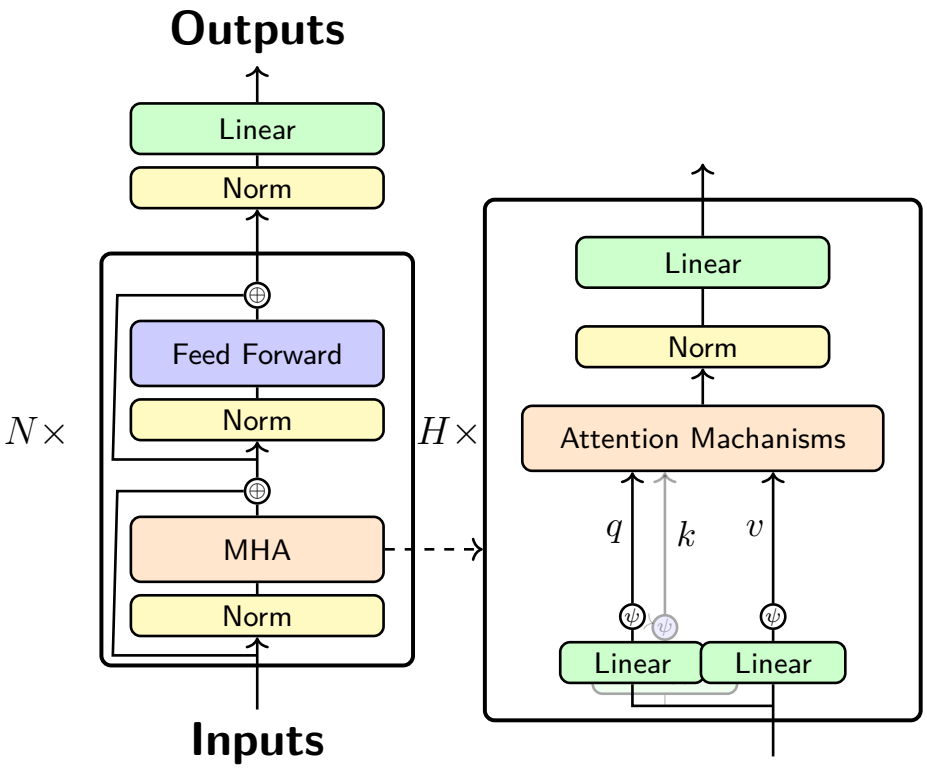


Figure: Auto-regressive Transformer architecture. (LLaMA3-style)

For a single head.

A **causally-masked softmax self-attention layer** maps the input sequence $\{x_t\}_{t=1}^T$, $x \in \mathbb{R}^{d_e}$ to an output sequence $\{o_t\}_{t=1}^T$, $o_t \in \mathbb{R}^{d_v}$ as:

$$\begin{aligned} k_t, q_t &= W_k x_t, W_q x_t \in \mathbb{R}^{d_k}, \\ v_t &= W_v x_t \in \mathbb{R}^{d_v}, \\ o_t &= \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v}. \end{aligned}$$

where $\kappa(k, q) = \exp(k^T q) > 0$ is the **softmax kernel**.

Transformers [Vas+23]

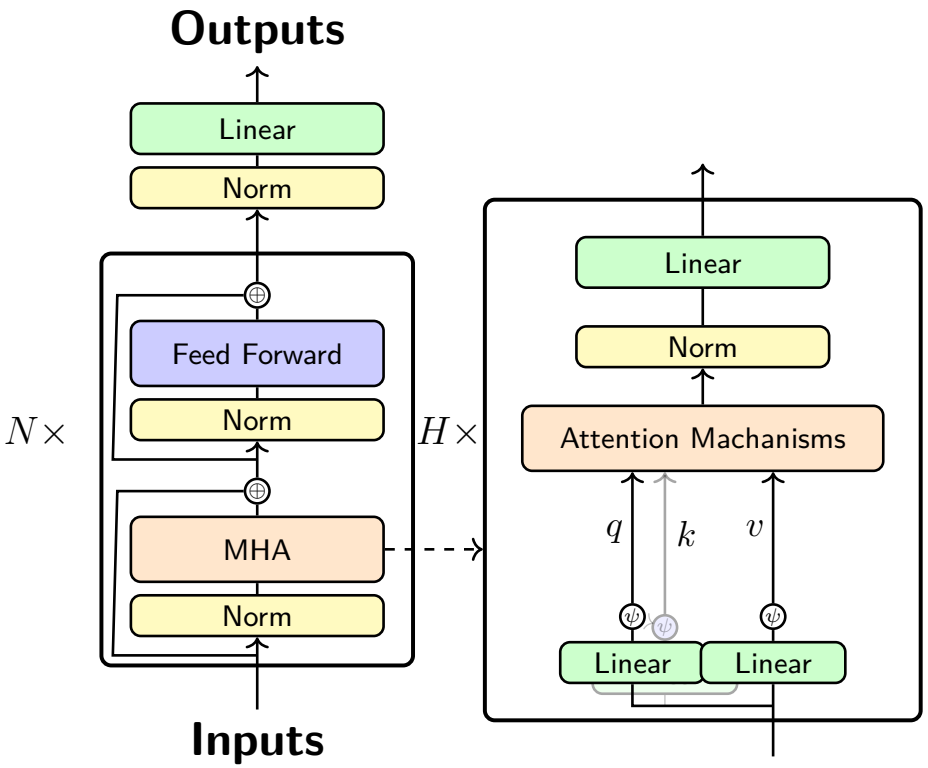


Figure: Auto-regressive Transformer architecture. (LLaMA3-style)

$$x_t = x_t + \Delta x_t^{\text{MHA}}$$

Multi-head attention (MHA) with H heads computes:

$$\Delta x_t^{\text{MHA}} = \sum_{h=1}^{H} P_h o_{h,t}$$

where h is head index, $P_h \in \mathbb{R}^{d \times d_v}$ is the projection matrix for head h.

Complexity Analysis

Notations: $x \in \mathbb{R}^{d_e}$, $q_t, k_t, \in \mathbb{R}^{d_k}$, $v_t, o_t \in \mathbb{R}^{d_v}$,
whole sequence of token embeddings $X \in \mathbb{R}^{T \times d_e}$
Q, K, V, $O \in \mathbb{R}^{T \times d}$

Complexity Analysis

Notations: $x \in \mathbb{R}^{d_e}$, $q_t, k_t, \in \mathbb{R}^{d_k}$, $v_t, o_t \in \mathbb{R}^{d_v}$, whole sequence of token embeddings $X \in \mathbb{R}^{T \times d_e}$
Q, K, V, $O \in \mathbb{R}^{T \times d}$

- d_e: dimension of token embedding
- d_k: dimension of key, query vectors
- d_v: dimension of value, output vectors
- T: sequence length

Complexity Analysis

Notations: $x \in \mathbb{R}^{d_e}$, $q_t, k_t, \in \mathbb{R}^{d_k}$, $v_t, o_t \in \mathbb{R}^{d_v}$,
whole sequence of token embeddings $X \in \mathbb{R}^{T \times d_e}$
Q, K, V, $O \in \mathbb{R}^{T \times d}$

- d_e: dimension of token embedding
- d_k: dimension of key, query vectors
- d_v: dimension of value, output vectors
- T: sequence length

Usually $d_e = 128 \sim 1024$, $d_{k/v} = 16 \sim 128$, $T = 256 \sim 32$k.
We assume $d_e = d_k = d_v = d$ in the following complexity analysis.

Softmax Attention

Softmax Attention with causal mask is a function mapping Q, K and V to O:

$$\begin{aligned} \text{(Parallel form:)} \quad O &= \text{softmax}(QK^T \odot L)V \in \mathbb{R}^{T \times d_v}, \\ \text{(Per-token form:)} \quad o_t &= \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v}, \end{aligned}$$

where $\kappa(k, q) = \exp(k^T q) > 0$ is the **softmax kernel** and L is a lower-triangular 1's matrix.

Softmax Attention

Softmax Attention with causal mask is a function mapping Q, K and V to O:

$$\text{(Parallel form:)} \quad O = \text{softmax}(QK^T \odot L)V \in \mathbb{R}^{T \times d_v},$$

$$\text{(Per-token form:)} \quad o_t = \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v},$$

where $\kappa(k, q) = \exp(k^T q) > 0$ is the **softmax kernel** and L is a lower-triangular 1's matrix.
Per-token form needs access to all past keys and values (KV Cache)

Complexity of Softmax Attention

$$
\begin{aligned}
Q, K, V &= XW_q^T, XW_k^T, XW_v^T, \in \mathbb{R}^{T \times d} \\
\text{(Parallel form:)} \quad O &= \text{softmax}(QK^T \odot L)V.
\end{aligned}
$$

- **Train**: Use Parallel form. Given T and $X \in \mathbb{R}^{T \times d_e}$,
  compute QK^T in $\mathcal{O}(T^2 d)$,
  compute softmax and multiply with V in $\mathcal{O}(T^2 d)$.
  Time complexity: $\mathcal{O}(T^2 d)$, Space complexity: $\mathcal{O}(T^2 + Td)$
  (due to QK^T)

Complexity of Softmax Attention

$$
\begin{aligned}
q_t, k_t, v_t &= W_q x_t, W_k x_t, W_v x_t \in \mathbb{R}^d \\
\text{(Per-token form:)} \quad o_t &= \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v}
\end{aligned}
$$

- **Inference**:
  At each step t, given $x_{1:t}$, $k_{1:t-1}$, $v_{1:t-1}$,
  Compute attention weights $\kappa(k_i, q_t)$ for $i = 1, \ldots, t$ in $\mathcal{O}(td)$,
  compute weighted sum in $\mathcal{O}(td)$.
  the Time complexity: $\mathcal{O}(td)$, Space complexity: $\mathcal{O}(td)$ (due to KV cache)

Complexity of Softmax Attention

$$\text{(Parallel form:)} \quad O = \text{softmax}(QK^T \odot L)V,$$

$$\text{(Per-token form:)} \quad o_t = \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v}$$

- Training: Time complexity: $\mathcal{O}(T^2 d)$, Space complexity: $\mathcal{O}(T^2 + Td)$ (due to QK^T)
- Inference: Time complexity: $\mathcal{O}(td)$, Space complexity: $\mathcal{O}(td)$ (due to KV cache)

Bottleneck: Quadratic time and space complexity during training; Linear space complexity during generation.

- **Try Linear Attention** for efficient long-context modeling.

$$\text{(Per-token form:)} \quad o_t = \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v}$$

Linear Attention [SIS21]

Softmax Attention:

$$\text{(Parallel form:)} \quad O = \text{softmax}(QK^T \odot L)V \in \mathbb{R}^{T \times d_v},$$

$$\text{(Per-token form:)} \quad o_t = \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v},$$

Linear Attention [SIS21]

Softmax Attention:

$$\text{(Parallel form:)} \quad O = \text{softmax}(QK^T \odot L)V \in \mathbb{R}^{T \times d_v},$$

$$\text{(Per-token form:)} \quad o_t = \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v},$$

Linear Attention:

$$\text{(Parallel form:)} \quad O = (QK^T \odot L)V,$$

$$\text{(Per-token form:)} \quad o_t = (\sum_{i=1}^{t} q_t k_i^T) v_i = \underbrace{(\sum_{i=1}^{t} v_i k_i^T)}_{:= M_t \in \mathbb{R}^{d_v \times d_k}} q_t.$$

Linear Attention [SIS21]

Softmax Attention:

$$\text{(Parallel form:)} \quad O = \text{softmax}(QK^T \odot L)V \in \mathbb{R}^{T \times d_v},$$

$$\text{(Per-token form:)} \quad o_t = \sum_{i=1}^{t} \frac{\kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)} v_i \in \mathbb{R}^{d_v},$$

Linear Attention:

$$\text{(Parallel form:)} \quad O = (QK^T \odot L)V,$$

$$\text{(Per-token form:)} \quad o_t = (\sum_{i=1}^{t} q_t k_i^T) v_i = \underbrace{(\sum_{i=1}^{t} v_i k_i^T)}_{:= M_t \in \mathbb{R}^{d_v \times d_k}} q_t.$$

M_t can be computed recurrently:

$$\text{(Recurrent form:)} \quad M_t = M_{t-1} + v_t k_t^T,$$

$$o_t = M_t q_t.$$

Linear Attention Layer as RNNs

Definition (Recurrent Neural Network)

A Recurrent Neural Network (RNN) maps an input sequence to an output sequence by maintaining a vector-valued hidden state.

$$\begin{aligned} h_t &= f_\theta(h_{t-1}, x_t) \in \mathbb{R}^{d_h}, \\ o_t &= g_\theta(h_t) \in \mathbb{R}^{d_o}. \end{aligned}$$

Linear Attention Layer as RNNs

Definition (Recurrent Neural Network)

A Recurrent Neural Network (RNN) maps an input sequence to an output sequence by maintaining a vector-valued hidden state.

$$\begin{aligned} h_t &= f_\theta(h_{t-1}, x_t) \in \mathbb{R}^{d_h}, \\ o_t &= g_\theta(h_t) \in \mathbb{R}^{d_o}. \end{aligned}$$

We often train RNNs using Backpropagation Through Time (BPTT), which requires to store all past value of hidden states $h_0, \ldots, h_t$.

Linear Attention Layer as RNNs

Definition (Recurrent Neural Network)

A Recurrent Neural Network (RNN) maps an input sequence to an output sequence by maintaining a vector-valued hidden state.

$$\begin{aligned} h_t &= f_\theta(h_{t-1}, x_t) \in \mathbb{R}^{d_h}, \\ o_t &= g_\theta(h_t) \in \mathbb{R}^{d_o}. \end{aligned}$$

We often train RNNs using Backpropagation Through Time (BPTT), which requires to store all past value of hidden states $h_0, \ldots, h_t$.

A Linear Attention layer can be viewed as a RNN maintaining a matrix-valued hidden state.

Linear Attention Layer as RNNs

Definition (Recurrent Neural Network)

A Recurrent Neural Network (RNN) maps an input sequence to an output sequence by maintaining a vector-valued hidden state.

$$\begin{aligned} h_t &= f_\theta(h_{t-1}, x_t) \in \mathbb{R}^{d_h}, \\ o_t &= g_\theta(h_t) \in \mathbb{R}^{d_o}. \end{aligned}$$

We often train RNNs using Backpropagation Through Time (BPTT), which requires to store all past value of hidden states $h_0, \ldots, h_t$.

A Linear Attention layer can be viewed as a RNN maintaining a matrix-valued hidden state.

BPTT requires to store $M_0, \ldots, M_T$. Space complexity: $\mathcal{O}(Td^2)$.

Complexity of Linear Attention

$$\text{(Parallel form:)} \quad O \quad = \quad (QK^T \odot L)V.$$

Due to mask $\odot L$ (non-associativity), training complexity is still quadratic.

Complexity of Linear Attention

$$\text{(Parallel form:)} \quad O = (QK^T \odot L)V.$$

Due to mask $\odot L$ (non-associativity), training complexity is still quadratic.

$$\begin{aligned} \text{(Recurrent form:)} \quad M_t &= M_{t-1} + v_t k_t^T, \\ o_t &= M_t q_t. \end{aligned}$$

- Lack of Parallelism in sequence dimension.
  Do not fully utilize GPUs.
- BPTT requires to store $M_0, \ldots, M_T$, which is memory inefficient.

Complexity of Linear Attention

$$\text{(Parallel form:)} \quad O = (QK^T \odot L)V.$$

Due to mask $\odot L$ (non-associativity), training complexity is still quadratic.

$$\begin{aligned} \text{(Recurrent form:)} \quad M_t &= M_{t-1} + v_t k_t^T, \\ o_t &= M_t q_t. \end{aligned}$$

- Lack of Parallelism in sequence dimension.
  Do not fully utilize GPUs.
- BPTT requires to store $M_0, \ldots, M_T$, which is memory inefficient.

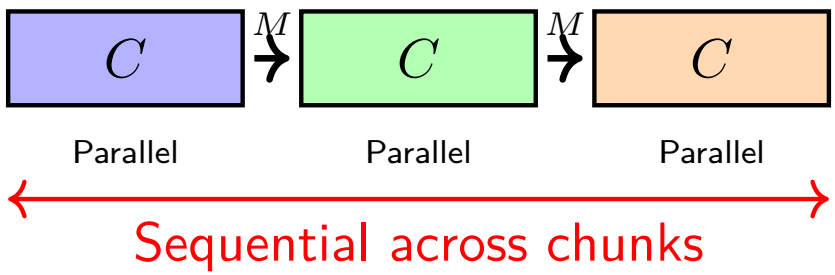
A pratical method to train it is **Chunkwise training**.

Chunkwise Training [Hua+22]

Popular method to train RNNs efficiently.
Split the whole sequence into chunks of length C.
For the $i-th$ chunk, denote $M_{[i]} := M_{iC}$,
and denote $\square_{[i]} := \square_{iC+1:(i+1)C}$ for submatrices of Q, K, V, O.

Chunkwise Training [Hua+22]

Popular method to train RNNs efficiently.
Split the whole sequence into chunks of length C.
For the $i-th$ chunk, denote $M_{[i]} := M_{iC}$,
and denote $\square_{[i]} := \square_{iC+1:(i+1)C}$ for submatrices of Q, K, V, O.



Chunkwise Training

$$\begin{aligned} \text{(Sequential update:)} \quad M_{[i+1]} &= M_{[i]} + V_{[i]}^T K_{[i]} \in \mathbb{R}^{d_v \times d_k} \\ \text{(Parallel compute:)} \quad O_{[i]} &= Q_{[i]} M_{[i]}^T + (Q_{[i]} K_{[i]}^T \odot L) V_{[i]}, \in \mathbb{R}^{C \times d_v} \end{aligned}$$

Chunkwise Training

$$\text{(Sequential update:)} \quad M_{[i+1]} = M_{[i]} + V_{[i]}^T K_{[i]} \in \mathbb{R}^{d_v \times d_k}$$

$$\text{(Parallel compute:)} \quad O_{[i]} = Q_{[i]} M_{[i]}^T + (Q_{[i]} K_{[i]}^T \odot L) V_{[i]}, \in \mathbb{R}^{C \times d_v}$$

Equivalence to recurrent form: For o_t in chunk i,

$$\begin{aligned} o_t^T &= q_t^T M_{iC}^T + \sum_{j=iC+1}^{t} q_t^T k_j v_j^T \\ &= \sum_{j=1}^{iC} q_t^T k_j v_j^T + \sum_{j=iC+1}^{t} q_t^T k_j v_j^T \\ &= \sum_{j=1}^{t} q_t^T k_j v_j^T. \end{aligned}$$

So $o_t = \sum_{j=1}^{t} q_t k_j^T v_j = M_t q_t$.

Chunkwise Training

Complexity: $\mathcal{O}(Td^2 + TCd)$ | $C \in \{16, 128, 256\}$ for Tensor Cores

Chunkwise Training

Complexity: $\mathcal{O}(Td^2 + TCd) \quad | \quad C \in \{16, 128, 256\}$ for Tensor Cores

It is reduced to parallel form if $C = T$; recurrent form if $C = 1$.

Update rule of M_t

Setup: Given input sequence $\{x_t\}_{t=1}^T$.
Linear Attention layer maps input to output $\{o_t\}_{t=1}^T$ as:

$$\begin{aligned} M_t &= M_{t-1} + v_t k_t^T, \\ o_t &= M_t q_t. \end{aligned}$$

Update rule of M_t

Setup: Given input sequence $\{x_t\}_{t=1}^T$.
Linear Attention layer maps input to output $\{o_t\}_{t=1}^T$ as:

$$\begin{aligned} M_t &= M_{t-1} + v_t k_t^T, \\ o_t &= M_t q_t. \end{aligned}$$

Online learning interpretation

Update rule for M_t from **online gradient descent** perspective:

Online learning interpretation

Update rule for M_t from **online gradient descent** perspective:

- Initialize: $M_0 = 0 \in \mathbb{R}^{d_v \times d_k}$, $\eta_t = 1, \forall t$.

Online learning interpretation

Update rule for M_t from **online gradient descent** perspective:

- Initialize: $M_0 = 0 \in \mathbb{R}^{d_v \times d_k}$, $\eta_t = 1, \forall t$.
- Key-value pair (k_t, v_t) is given at step t,

Online learning interpretation

Update rule for M_t from **online gradient descent** perspective:

- Initialize: $M_0 = 0 \in \mathbb{R}^{d_v \times d_k}$, $\eta_t = 1, \forall t$.
- Key-value pair (k_t, v_t) is given at step t,
- Objective at step t is given by $L(M; k_t, v_t) := -\langle Mk_t, v_t \rangle$.

Online learning interpretation

Update rule for M_t from **online gradient descent** perspective:

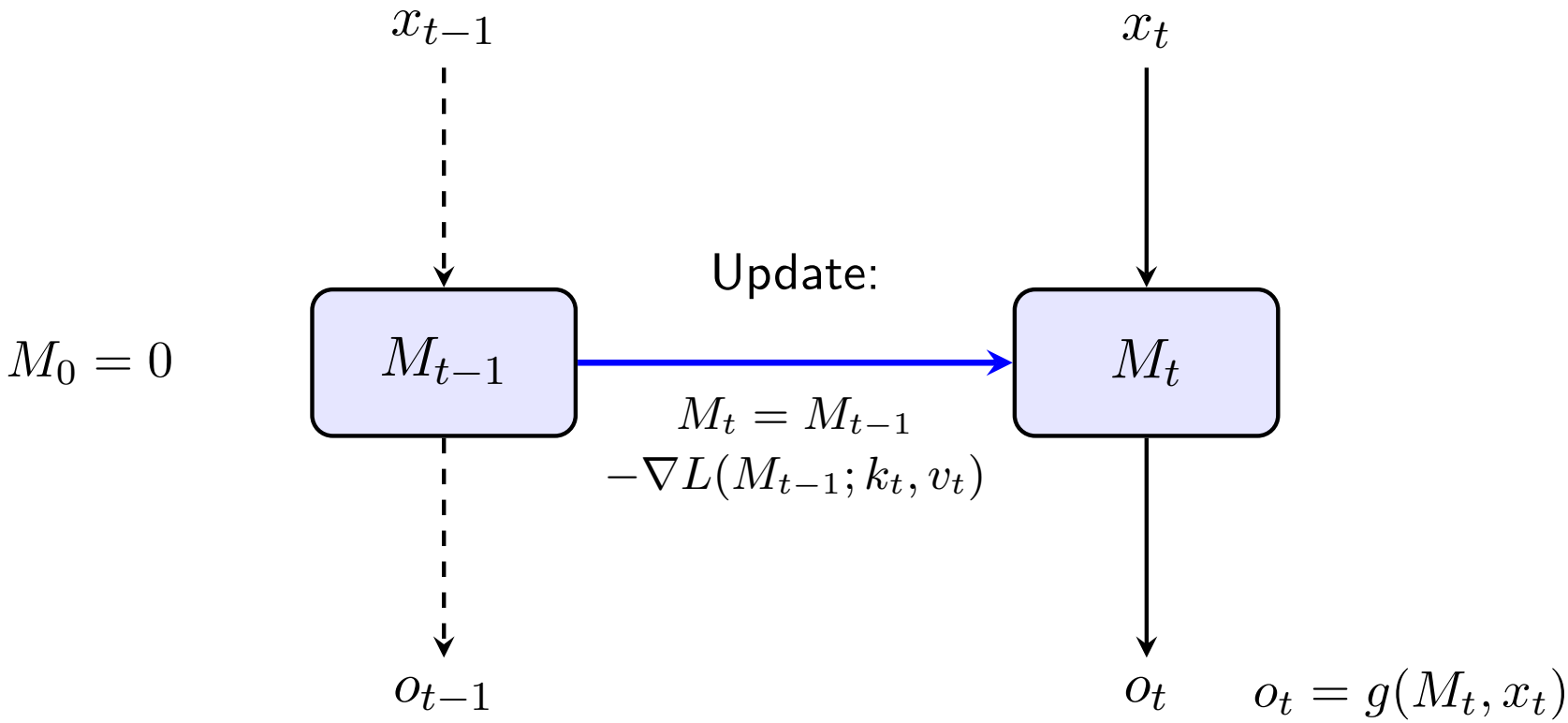
- Initialize: $M_0 = 0 \in \mathbb{R}^{d_v \times d_k}$, $\eta_t = 1, \forall t$.
- Key-value pair (k_t, v_t) is given at step t,
- Objective at step t is given by $L(M; k_t, v_t) := -\langle Mk_t, v_t \rangle$.
- Online gradient descent:
  $M_t = M_{t-1} - \eta_{t-1} \nabla L(M_{t-1}; k_t, v_t) = M_{t-1} - 1 \cdot (-v_t k_t^T)$.
  $\implies M_t = M_{t-1} + v_t k_t^T$.

Online learning interpretation

Update rule for M_t from **online gradient descent** perspective:

- Initialize: $M_0 = 0 \in \mathbb{R}^{d_v \times d_k}$, $\eta_t = 1, \forall t$.
- Key-value pair (k_t, v_t) is given at step t,
- Objective at step t is given by $L(M; k_t, v_t) := -\langle Mk_t, v_t \rangle$.
- Online gradient descent:
  $M_t = M_{t-1} - \eta_{t-1} \nabla L(M_{t-1}; k_t, v_t) = M_{t-1} - 1 \cdot (-v_t k_t^T)$.
  $\implies M_t = M_{t-1} + v_t k_t^T$.

Test Time Training (TTT) [Sun+25]: compress context into **fixed-size** states M_t via optimizing 'internal objective'.


x_{t-1}
x_t
Update:
$M_0 = 0$
M_{t-1}
M_t
$M_t = M_{t-1}$
$-\nabla L(M_{t-1}; k_t, v_t)$
o_{t-1}
o_t
$o_t = g(M_t, x_t)$

Fast Weights

Further remark after presentation: The name of FWP is not important. Please omit this page. **Fast weights**: Input-dependent weights updated at test time.

Fast Weights

Further remark after presentation: The name of FWP is not important. Please omit this page. **Fast weights**: Input-dependent weights updated at test time.

Fast Weight Programmers: Let the slow net **learn to program** the fast net via gradient descent.

Fast Weights

Further remark after presentation: The name of FWP is not important. Please omit this page. **Fast weights**: Input-dependent weights updated at test time.

Fast Weight Programmers: Let the slow net **learn to program** the fast net via gradient descent.
Example [Sch92]: Given input sequence $\{x_t\}_{t=1}^{T}$, the model map to output sequence $\{o_t\}_{t=1}^{T}$ as:

Fast Weights

Further remark after presentation: The name of FWP is not important. Please omit this page. **Fast weights**: Input-dependent weights updated at test time.

Fast Weight Programmers: Let the slow net **learn to program** the fast net via gradient descent.
Example [Sch92]: Given input sequence $\{x_t\}_{t=1}^T$, the model map to output sequence $\{o_t\}_{t=1}^T$ as:

$$\begin{aligned} a_t, b_t &= W_a x_t, W_b x_t \\ W_t &= \sigma(W_{t-1} + a_t b_t^T) \\ o_t &= W_t x_t. \end{aligned}$$

Slow weights W_a, W_b are trainable; Fast weights W_t .
Instructions

Linear Attention as Fast Weight Programmer(FWP) [SIS21]

Further remark after presentation: The name of FWP is not important. Please omit this page.

$$k_t, v_t, q_t = W_k x_t, W_v x_t, W_q x_t$$
$$M_t = M_{t-1} + v_t k_t^T$$
$$o_t = M_t q_t$$

Current Limitations of Linear Attention

1. Memory capacity is limited by d_k for error-free *retrieval*.
2. Cannot dynamically *forget* what it stores.

Current Limitations of Linear Attention

1. Memory capacity is limited by d_k for error-free *retrieval*.
2. Cannot dynamically *forget* what it stores.

how to improve it?

We desgin a reccurent layer from 2 major perspectives:

We desgin a reccurent layer from 2 major perspectives:

1. Hidden States M: Lift the dimension of M (e.g. kernel trick, deep MLP). (More Expressive Power).

We desgin a reccurent layer from 2 major perspectives:

1. Hidden States M: Lift the dimension of M (e.g. kernel trick, deep MLP). (More Expressive Power).
2. Update Rules for M: design objectives (e.g. least-square) and optimizer (e.g. gradient descent, momentum) for M. (To Better Memorize Context).

Delta Rule [SIS21]

- Objective: $L(M; k_t, v_t) := \frac{1}{2}\|Mk_t - v_t\|_2^2$.

Delta Rule [SIS21]

- Objective: $L(M; k_t, v_t) := \frac{1}{2}\|Mk_t - v_t\|_2^2$.
- Online gradient descent:
  $\nabla L(M_{t-1}; k_t, v_t) = \frac{1}{2}(M_{t-1}k_t - v_t)^T(M_{t-1}k_t - v_t) = (M_{t-1}k_t - v_t)k_t^T$.

Delta Rule [SIS21]

- Objective: $L(M; k_t, v_t) := \frac{1}{2}\|Mk_t - v_t\|_2^2$.
- Online gradient descent:
  $\nabla L(M_{t-1}; k_t, v_t) = \frac{1}{2}(M_{t-1}k_t - v_t)^T(M_{t-1}k_t - v_t) =$
  $(M_{t-1}k_t - v_t)k_t^T$.
  $M_t = M_{t-1} - \eta_t \nabla L(M_{t-1}; k_t, v_t) = M_{t-1} + \eta_t(v_t - M_{t-1}k_t)k_t^T$.

Delta Rule [SIS21]

- Objective: $L(M; k_t, v_t) := \frac{1}{2}\|Mk_t - v_t\|_2^2$.
- Online gradient descent:
  $\nabla L(M_{t-1}; k_t, v_t) = \frac{1}{2}(M_{t-1}k_t - v_t)^T(M_{t-1}k_t - v_t) = (M_{t-1}k_t - v_t)k_t^T$.
  $M_t = M_{t-1} - \eta_t \nabla L(M_{t-1}; k_t, v_t) = M_{t-1} + \eta_t(v_t - M_{t-1}k_t)k_t^T$.
  Set $\eta_t = \beta_t \in (0, 1)$,

Delta Rule [SIS21]

- Objective: $L(M; k_t, v_t) := \frac{1}{2}\|Mk_t - v_t\|_2^2$.
- Online gradient descent:
  $\nabla L(M_{t-1}; k_t, v_t) = \frac{1}{2}(M_{t-1}k_t - v_t)^T(M_{t-1}k_t - v_t) = (M_{t-1}k_t - v_t)k_t^T$.
  $M_t = M_{t-1} - \eta_t \nabla L(M_{t-1}; k_t, v_t) = M_{t-1} + \eta_t(v_t - M_{t-1}k_t)k_t^T$.
  Set $\eta_t = \beta_t \in (0, 1)$,

$$\implies M_t = M_{t-1}(I - \beta_t k_t k_t^T) + \beta_t v_t k_t^T. \quad \text{(Delta Rule)}$$

  where $\beta_t = \sigma(W_\beta x_t) \in (0, 1)$ is a **data-dependent** input gate.

Delta Rule [SIS21]

Proximal interpretation:

$$M_t = \arg\min_M \left\{ \langle (M_{t-1}k_t - v_t)k_t^T, M - M_{t-1} \rangle + \frac{1}{2\eta_t} \|M - M_{t-1}\|_F^2 \right\}$$

The Architecture of DeltaNet [Yan+]

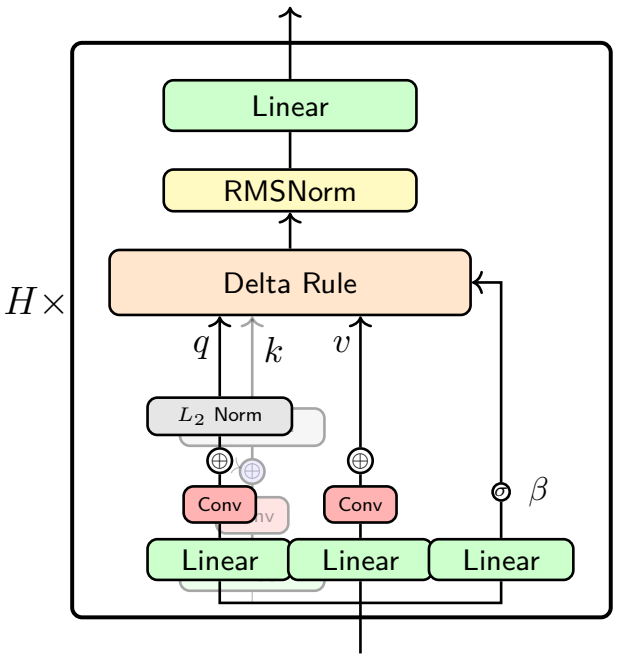


Figure: DeltaNet.

$$
\begin{aligned}
q_t^h,\ k_t^h &= \mathrm{L2Norm}\Big(\mathrm{SiLU}\Big(\mathrm{ShortConv}(W_{q/k}^h\,\mathbf{x}_t)\Big)\Big)\\
v_t^h &= \mathrm{SiLU}\Big(\mathrm{ShortConv}(W_v^h\,\mathbf{x}_t)\Big),\\
\beta_t^h &= \mathrm{Sigmoid}\Big(W_\beta^h\,\mathbf{x}_t\Big),\\
o_t &= W_o\Big(\mathrm{Norm}\Big(\mathrm{DeltaRule}(\cdot)\Big)\Big).
\end{aligned}
$$

where $h = 1, ..., H$ represents the head index.

Add Decay Term

Mamba2 [DG24]: $M_t = \alpha_t M_{t-1} + v_t k_t^T$.
where $\alpha_t \in (0, 1)$ is a **data-dependent** scalar-valued forget gate.

Gated DeltaNet (GDN) [YKH25]

Add decay into Delta Rule:
$M_t = \alpha_t M_{t-1}(I - \beta_t k_t k_t^T) + \beta_t v_t k_t^T$.
where $\alpha_t \in (0, 1)$ is a **data-dependent** scalar-valued forget gate and $\beta_t \in (0, 1)$ is an input gate.

Gated DeltaNet (GDN) [YKH25]

Add decay into Delta Rule:

$M_t = \alpha_t M_{t-1}(I - \beta_t k_t k_t^T) + \beta_t v_t k_t^T$.

where $\alpha_t \in (0, 1)$ is a **data-dependent** scalar-valued forget gate and $\beta_t \in (0, 1)$ is an input gate.

	S-NIAH-1 (pass-key retrieval)				S-NIAH-2 (number in haystack)				S-NIAH-3 (uuid in haystack)		
Model	1K	2K	4K	8K	1K	2K	4K	8K	1K	2K	4K
DeltaNet	97.4	96.8	**99.0**	**98.8**	98.4	45.6	18.6	14.4	85.2	47.0	22.4
Mamba2	**99.2**	**98.8**	65.4	30.4	99.4	98.8	56.2	17.0	64.4	47.6	4.6
Gated DeltaNet	98.4	88.4	91.4	91.8	**100.0**	**99.8**	**92.2**	**29.6**	**86.6**	**84.2**	**27.6**

1. Decay hurts memory retention but facilitates filtering.
2. Delta rule helps memorization.

S-NIAH-1: Synthetic context (repeated tokens)

Context: `[AAA AAA AAA ...` `city:Paris` `... AAA AAA]`

Query: `city?` $\rightarrow$ *Answer:* `Paris`

Tests: Long-term memory retention

S-NIAH-1: Synthetic context (repeated tokens)

Context: [AAA AAA AAA ... city:Paris ... AAA AAA]
Query: city? → Answer: Paris

Tests: Long-term memory retention

S-NIAH-2: Real-world essay context

Context: [The quick brown fox ... password:7492 ... jumps over ...]
Query: password? → Answer: 7492

Tests: Filtering irrelevant information

S-NIAH-1: Synthetic context (repeated tokens)

Context: [AAA AAA AAA ... city:Paris ... AAA AAA]
Query: city? → Answer: Paris

Tests: Long-term memory retention

S-NIAH-2: Real-world essay context

Context: [The quick brown fox ... password:7492 ... jumps over ...]
Query: password? → Answer: 7492

Tests: Filtering irrelevant information

S-NIAH-3: Complex value (UUID)

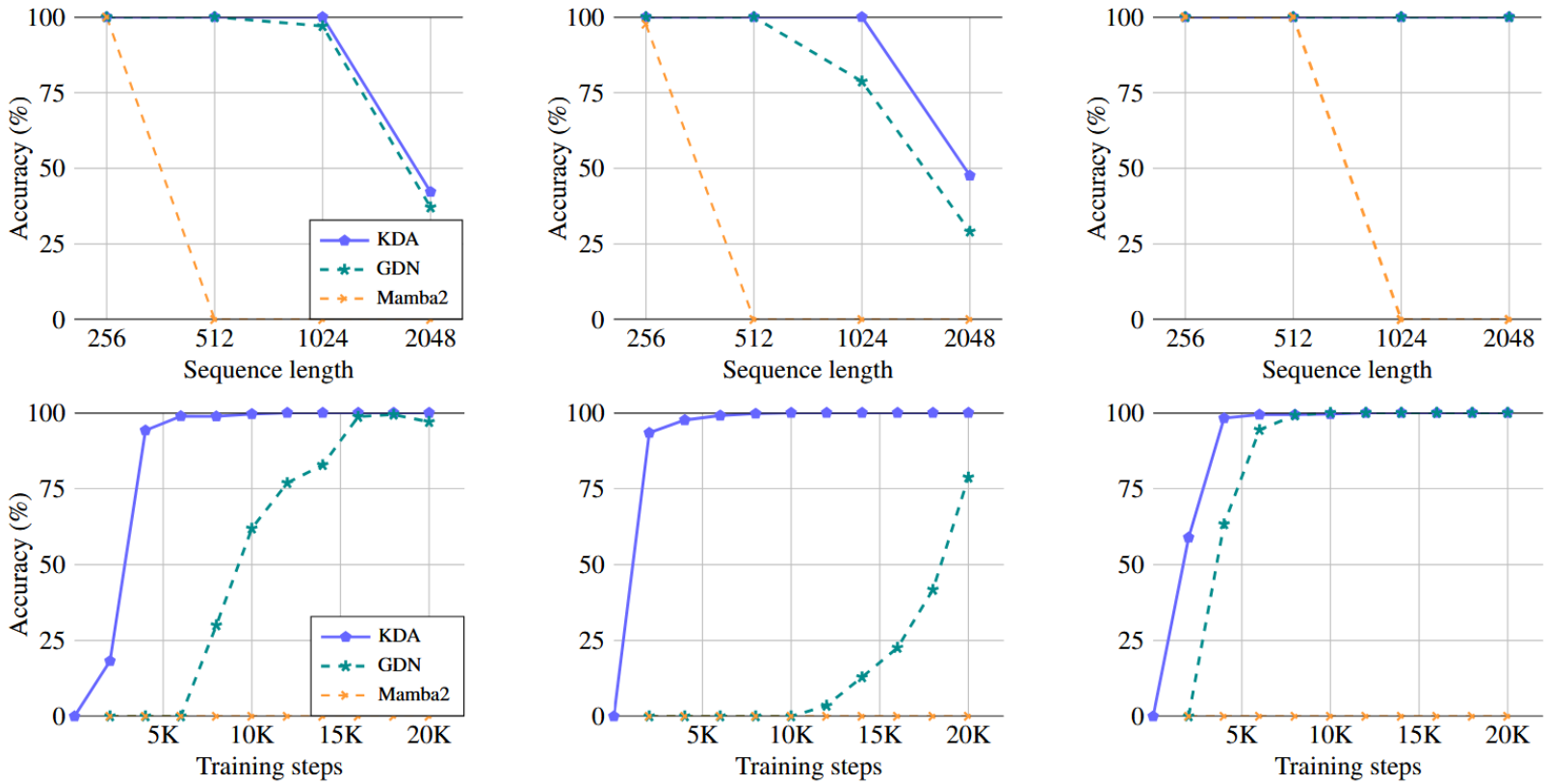
Context: [Essay text ... id:a3f2-b8c1-9d4e ... more text ...]
Query: id? → Answer: a3f2-b8c1-9d4e

Tests: Complex pattern memorization

Kimi Delta Attention (KDA) [Tea+25]

Replace the scalar-valued forget gate α_t with a fine-grained diagonalized gate:

$M_t = M_{t-1}\text{Diag}(\alpha_t)(I - \beta_t k_t k_t^T) + \beta_t v_t k_t^T$, where $\alpha_t \in (0,1)^{d_k}$.



(a) Palindrome (b) MQAR (c) Stack

Kimi Delta Attention (KDA) [Tea+25]

MQAR:

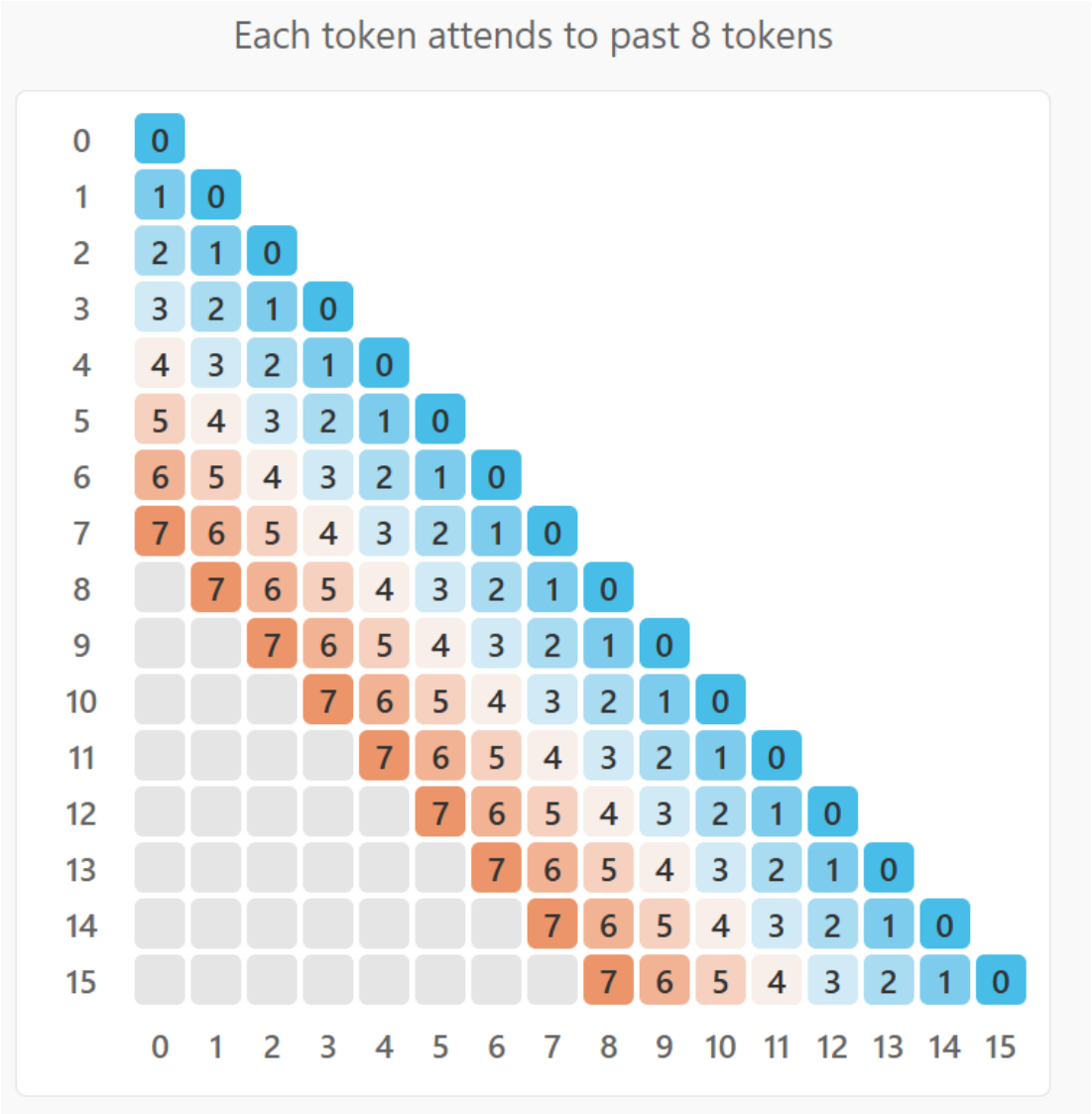
Input	A	1	C	3	B	0	M	8	G	5	E	4	<SEP>	B	G
Output	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	ϕ	0	5

Stack:
Push i E Pop i E

Sliding Window Attention (SWA)

is a kind of *sparse attention*.

$(QK^T \odot L)$ where mask L is a band matrix with bandwidth w:

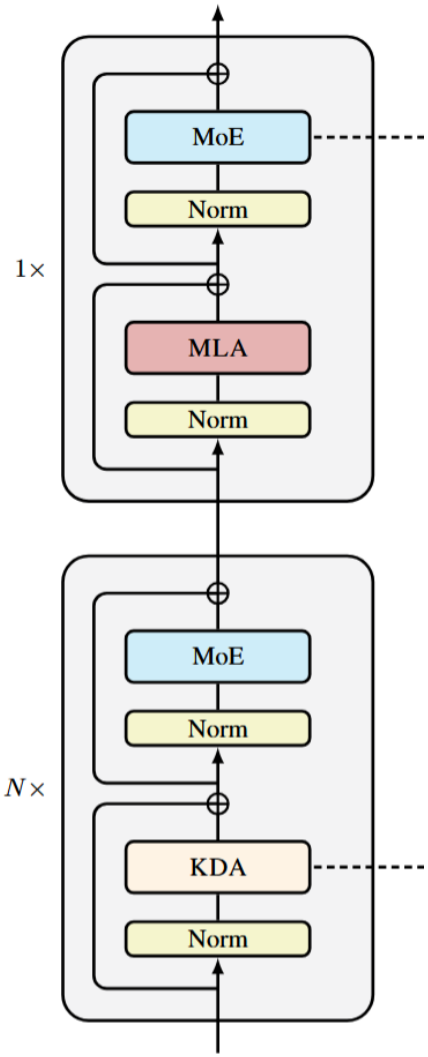


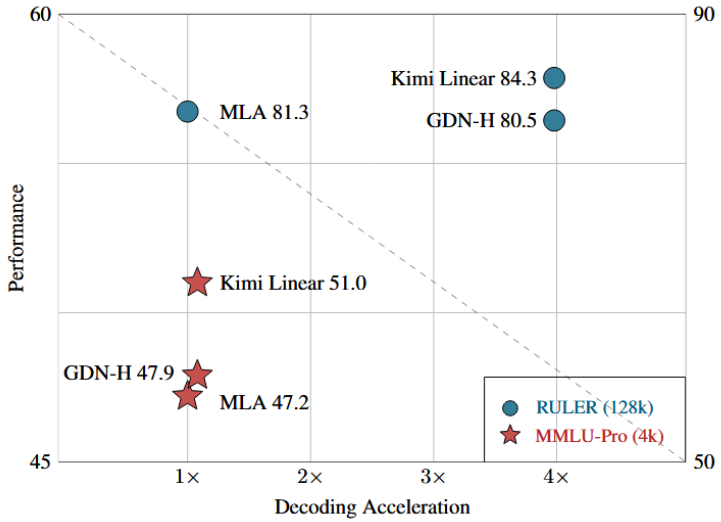
Hybird Attention

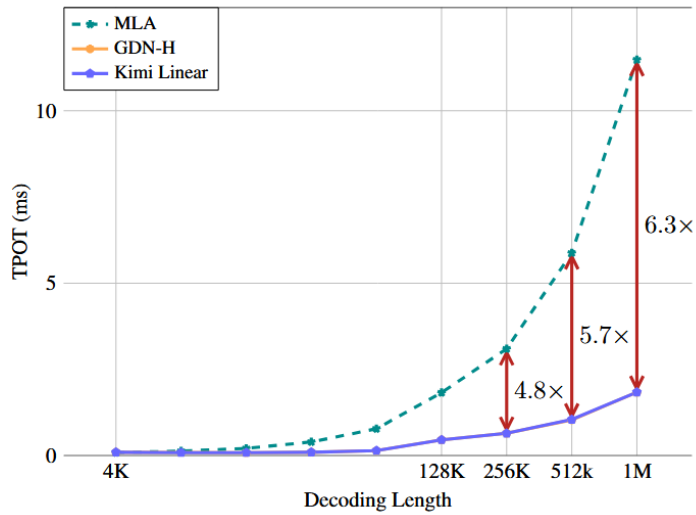
Hybird Attention architecture interleaves linear or sparse attention layers with full attention layers.
For example:

- Kimi Linear [Tea+25] interleaves KDA with Multi-Head Latent Attention at a 3:1 ratio.
- GPT-OSS [Ope25] interleaves SWA ($w = 128$) with Grouped-Query Attention at a 1:1 ratio.

Hybird Attention: Kimi Linear







1. Performance. Kimi Linear outperforms MLA and GDN-H on two benchmarks (reasoning and long-context) with strict fair comparisons with 1.4T training tokens.
2. Efficiency. Kimi Linear (blue line) maintains a low Time per output token (TPOT), outperforming MLA at long sequences.

Hybird Attention: Disadvantages

Despite hybird attention model has good performance compared to full attention on specific benchmarks,

1. the model had clear deficits in complex reasoning tasks.
2. those proxy benchmarks may not reflect real-world downstream performance at larger scales.

`https://huggingface.co/blog/MiniMax-AI/why-did-m2-end-up-as-a-full-attention-model`

Contents

Softmax Attention via Kernel Trick

$$o_t = \sum_{i=1}^{t} \frac{v_i \kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)}, \qquad \kappa(k, q) = \exp(k^T q)$$

Kernel $\Leftrightarrow$ Feature Map: $\kappa(k, q) = \langle \phi(k), \phi(q) \rangle$

Softmax Attention via Kernel Trick

$$o_t = \sum_{i=1}^{t} \frac{v_i \kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)}, \qquad \kappa(k, q) = \exp(k^T q)$$

Kernel $\Leftrightarrow$ **Feature Map**: $\kappa(k, q) = \langle \phi(k), \phi(q) \rangle$

For softmax kernel: $\phi^*(x) = \left(1; \frac{x}{\sqrt{1!}}; \frac{x^{\otimes 2}}{\sqrt{2!}}; \cdots\right)$,

infinite-dimensional where $x^{\otimes p} = \underbrace{x \otimes x \otimes \cdots \otimes x}_{p \text{ times}}$.

Softmax Attention via Kernel Trick

$$o_t = \sum_{i=1}^{t} \frac{v_i \kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)}, \quad \kappa(k, q) = \exp(k^T q)$$

Kernel $\Leftrightarrow$ **Feature Map**: $\kappa(k, q) = \langle \phi(k), \phi(q) \rangle$

For softmax kernel: $\phi^*(x) = \left(1; \frac{x}{\sqrt{1!}}; \frac{x^{\otimes 2}}{\sqrt{2!}}; \cdots\right)$,

infinite-dimensional where $x^{\otimes p} = \underbrace{x \otimes x \otimes \cdots \otimes x}_{p \text{ times}}$.

$y \otimes y = \text{vec}(yy^T)$,

Softmax Attention via Kernel Trick

$$o_t = \sum_{i=1}^{t} \frac{v_i \kappa(k_i, q_t)}{\sum_{j=1}^{t} \kappa(k_j, q_t)}, \quad \kappa(k, q) = \exp(k^T q)$$

Kernel $\Leftrightarrow$ **Feature Map**: $\kappa(k, q) = \langle \phi(k), \phi(q) \rangle$

For softmax kernel: $\phi^*(x) = \left(1; \frac{x}{\sqrt{1!}}; \frac{x^{\otimes 2}}{\sqrt{2!}}; \cdots\right)$,
infinite-dimensional where $x^{\otimes p} = \underbrace{x \otimes x \otimes \cdots \otimes x}_{p \text{ times}}$.

$y \otimes y = \text{vec}(yy^T)$,

$$\begin{aligned}(x \otimes x)^T (y \otimes y) &= (x \otimes x)^T \text{vec}(yy^T) \\ &\overset{(*)}{=} x^T (yy^T) x \\ &= (x^T y)^2.\end{aligned}$$

where (*) is due to $(L \otimes R)\text{vec}(G) = \text{vec}(RGL^T)$

Linear Attention: Replace ϕ^* with finite dimemsional $\phi : \mathbb{R}^d \to \mathbb{R}^{d'}$

$$y_t = \frac{(\sum_{i=1}^{t} v_i \phi(k_i)^T)\phi(q_t)}{(\sum_{j=1}^{t} \phi(k_j))^T \phi(q_t)} \quad \Rightarrow \quad \text{Recurrent form!}$$

Polynomial Feature Map [Beh+25]

Goal: Approximate softmax kernel with finite-dimensional features

Softmax kernel (infinite Taylor series):

$$\exp(q^T k) = \sum_{m=0}^{\infty} \frac{(q^T k)^m}{m!} = \phi^*(q)^T \phi^*(k)$$

Polynomial Feature Map [Beh+25]

Goal: Approximate softmax kernel with finite-dimensional features

Softmax kernel (infinite Taylor series):

$$\exp(q^T k) = \sum_{m=0}^{\infty} \frac{(q^T k)^m}{m!} = \phi^*(q)^T \phi^*(k)$$

Polynomial approximation (truncated + learnable):

$$\exp(q^T k) \approx \phi_p(q)^T \phi_p(k) = \sum_{m=0}^{p} a_m (q^T k)^m$$

where $a_m \in \mathbb{R}$ are **learnable** coefficients, initialized as $a_m = \frac{1}{m!}$.

Polynomial Feature Map [Beh+25]

Goal: Approximate softmax kernel with finite-dimensional features

Softmax kernel (infinite Taylor series):

$$\exp(q^T k) = \sum_{m=0}^{\infty} \frac{(q^T k)^m}{m!} = \phi^*(q)^T \phi^*(k)$$

Polynomial approximation (truncated + learnable):

$$\exp(q^T k) \approx \phi_p(q)^T \phi_p(k) = \sum_{m=0}^{p} a_m (q^T k)^m$$

where $a_m \in \mathbb{R}$ are **learnable** coefficients, initialized as $a_m = \frac{1}{m!}$.

Advantages:

- Finite dimension $\Rightarrow$ efficient computation
- Learnable $a_m \Rightarrow$ adapt to data (not fixed Taylor coefficients)
- Higher $p \Rightarrow$ better approximation, higher memory capacity

Contents

Limitations of Linear Attention

1. **Limited capacity**: Only d_k orthogonal keys storable
2. **Suboptimality**: Online update only memorizes current token (v_t, k_t).
3. **Optimizer**: Gradient descent.

Limitations of Linear Attention

1. **Limited capacity**: Only d_k orthogonal keys storable
2. **Suboptimality**: Online update only memorizes current token (v_t, k_t).
3. **Optimizer**: Gradient descent.

Approches [Beh+25]:

1. Polynomial feature map $\phi_p(x) \Rightarrow$ higher capacity

Limitations of Linear Attention

1. **Limited capacity**: Only d_k orthogonal keys storable
2. **Suboptimality**: Online update only memorizes current token (v_t, k_t).
3. **Optimizer**: Gradient descent.

Approches [Beh+25]:

1. Polynomial feature map $\phi_p(x) \Rightarrow$ higher capacity
2. deep MLP $M(\cdot) \Rightarrow$ more expressive power

$$M(\cdot) \quad = \quad (\cdot) + W_1(\sigma(W_2(\cdot)) \otimes W_3(\cdot)).$$

Limitations of Linear Attention

1. **Limited capacity**: Only d_k orthogonal keys storable
2. **Suboptimality**: Online update only memorizes current token (v_t, k_t).
3. **Optimizer**: Gradient descent.

Approches [Beh+25]:

1. Polynomial feature map $\phi_p(x) \Rightarrow$ higher capacity
2. deep MLP $M(\cdot) \Rightarrow$ more expressive power

$$M(\cdot) \quad = \quad (\cdot) + W_1(\sigma(W_2(\cdot)) \otimes W_3(\cdot)).$$

3. Sliding window loss help memorize local context

Limitations of Linear Attention

1. **Limited capacity**: Only d_k orthogonal keys storable
2. **Suboptimality**: Online update only memorizes current token (v_t, k_t).
3. **Optimizer**: Gradient descent.

Approches [Beh+25]:

1. Polynomial feature map $\phi_p(x) \Rightarrow$ higher capacity
2. deep MLP $M(\cdot) \Rightarrow$ more expressive power

$$M(\cdot) \quad = \quad (\cdot) + W_1(\sigma(W_2(\cdot)) \otimes W_3(\cdot)).$$

3. Sliding window loss help memorize local context

$$\sum_{i=t-w+1}^{t} \eta_i^{(t)} \| M(\phi(k_i)) - v_i \|_2^2.$$

where $\eta_i^{(t)} \in (0, 1)$ is a input-dependent decay term.

Limitations of Linear Attention

1. **Limited capacity**: Only d_k orthogonal keys storable
2. **Suboptimality**: Online update only memorizes current token (v_t, k_t).
3. **Optimizer**: Gradient descent.

Approches [Beh+25]:

1. Polynomial feature map $\phi_p(x) \Rightarrow$ higher capacity
2. deep MLP $M(\cdot) \Rightarrow$ more expressive power

$$M(\cdot) \quad = \quad (\cdot) + W_1(\sigma(W_2(\cdot)) \otimes W_3(\cdot)).$$

3. Sliding window loss help memorize local context

$$\sum_{i=t-w+1}^{t} \eta_i^{(t)} \| M(\phi(k_i)) - v_i \|_2^2.$$

where $\eta_i^{(t)} \in (0, 1)$ is a input-dependent decay term.

4. Muon optimizer

ATLAS: Locally Optimal High-Capacity Memory [Beh+25]

Key Ideas: Sliding window + Muon optimizer + deep MLP memory

$$S_t = \theta_t S_{t-1} + \nabla \sum_{i=t-w+1}^{t} \eta_i^{(t)} \|M(\phi(k_i)) - v_i\|_2^2$$

$$M_t = \alpha_t M_{t-1} - \eta_t \,\text{NewtonShulz-}k(S_t)$$

where MLP M_t's parameters are updated.

ATLAS: Locally Optimal High-Capacity Memory [Beh+25]

Key Ideas: Sliding window + Muon optimizer + deep MLP memory

$$S_t = \theta_t S_{t-1} + \nabla \sum_{i=t-w+1}^{t} \eta_i^{(t)} \|M(\phi(k_i)) - v_i\|_2^2$$

$$M_t = \alpha_t M_{t-1} - \eta_t \,\text{NewtonShulz-}k(S_t)$$

where MLP M_t's parameters are updated.

- NewtonShulz-$1(S) = aS + bS^3 + cS^5$ iteratively approximate the matrix sign of its initial input matrix.
- Parallelizable using Chunkwise training (omit details).

ATLAS: Locally Optimal High-Capacity Memory [Beh+25]

Key Ideas: Sliding window + Muon optimizer + deep MLP memory

$$S_t = \theta_t S_{t-1} + \nabla \sum_{i=t-w+1}^{t} \eta_i^{(t)} \|M(\phi(k_i)) - v_i\|_2^2$$

$$M_t = \alpha_t M_{t-1} - \eta_t \,\text{NewtonShulz-}k(S_t)$$

where MLP M_t's parameters are updated.

- NewtonShulz-$1(S) = aS + bS^3 + cS^5$ iteratively approximate the matrix sign of its initial input matrix.
- Parallelizable using Chunkwise training (omit details).

Experiments Results of ATLAS

Results: 1. Ablation studies validate each component's effectiveness
2. Outperforms Transformer on most benchmarks

Experiments Results of ATLAS

Results: 1. Ablation studies validate each component's effectiveness
2. Outperforms Transformer on most benchmarks
3. Gap: Still lags on in-context recall tasks

Contents

Model Weight Matrices are also Compressing 'Context'

Example. (A Simple Example of MLP Training)
We consider a 1-layer MLP with input $x_t \in \mathbb{R}^d$ and output $y_t = W_t x_t$ and ground-truth label $z_t \in \mathbb{R}^k$ at step t.
Objective and Gradient Descent Update

$$\begin{aligned} W^* &= \arg\min_W \mathcal{L}(W; \{x_i, z_i\}_i), \\ W_{t+1} &= W_t - \eta_{t+1} \nabla_{W_t} \mathcal{L}(W_t; x_{t+1}) \\ &= W_t - \eta_{t+1} \nabla_{y_{t+1}} \mathcal{L}(W_t; x_{t+1}) \cdot x_{t+1}^T \text{ by chain rule,} \end{aligned}$$

where $y_{t+1} = W_t x_{t+1}$.

Equivalent Associative-Memory Optimization View

$$W_{t+1} = \arg\min_{W} \left\langle W x_{t+1}, \nabla_{y_{t+1}} \mathcal{L}(W_t; x_{t+1}) \right\rangle + \frac{1}{2\eta_{t+1}} \|W - W_t\|_F^2$$

During training, the slow weights similarly compress "key-value" pairs.

Nested MLP layers

Motivation: Different knowledge updates at different frequencies

- High frequency f_l: short-term, updates often
- Low frequency f_l: long-term, updates rarely

Nested MLP layers

Motivation: Different knowledge updates at different frequencies

- High frequency f_l: short-term, updates often
- Low frequency f_l: long-term, updates rarely

Architecture: Chain of MLPs with different update frequencies $(f_1 > f_2 > \cdots > f_k)$

$$y_t = \text{MLP}^{(f_k)}\left(\text{MLP}^{(f_{k-1})}\left(\cdots \text{MLP}^{(f_1)}(x_t)\right)\right)$$

Nested MLP layers

Motivation: Different knowledge updates at different frequencies

- High frequency f_l: short-term, updates often
- Low frequency f_l: long-term, updates rarely

Architecture: Chain of MLPs with different update frequencies $(f_1 > f_2 > \cdots > f_k)$

$$y_t = \text{MLP}^{(f_k)}\left(\text{MLP}^{(f_{k-1})}\left(\cdots \text{MLP}^{(f_1)}(x_t)\right)\right)$$

Update Rule: Parameters $\theta^{(f_l)}$ updated every $C^{(l)} = \frac{\max_l C^{(l)}}{f_l}$ steps: Here $C^{(l)} = \frac{\max_l C^{(l)}}{f_l}$ is not clearly defined in the paper. Just treat it as constant for each MLP layer.

Nested MLP layers

Motivation: Different knowledge updates at different frequencies

- High frequency f_l: short-term, updates often
- Low frequency f_l: long-term, updates rarely

Architecture: Chain of MLPs with different update frequencies $(f_1 > f_2 > \cdots > f_k)$

$$y_t = \text{MLP}^{(f_k)}\left(\text{MLP}^{(f_{k-1})}\left(\cdots \text{MLP}^{(f_1)}(x_t)\right)\right)$$

Update Rule: Parameters $\theta^{(f_l)}$ updated every $C^{(l)} = \frac{\max_l C^{(l)}}{f_l}$ steps: Here $C^{(l)} = \frac{\max_l C^{(l)}}{f_l}$ is not clearly defined in the paper. Just treat it as constant for each MLP layer.

$$\theta_{i+1}^{(f_l)} = \theta_i^{(f_l)} - \begin{cases} \sum_{t=i-C^{(l)}}^{i} \eta_t^{(l)} \nabla \mathcal{L}(\theta_t^{(f_l)}; x_t) & \text{if } i \equiv 0 \pmod{C^{(l)}} \\ 0 & \text{otherwise} \end{cases}$$

Nested MLP layers

Interpretation:

- $\theta^{(f_l)}$ compresses context of size $C^{(l)}$ into parameters
- Standard Transformer: $k = 1$

References I

[Beh+25] Ali Behrouz et al. *ATLAS: Learning to Optimally Memorize the Context at Test Time*. 2025. DOI: 10.48550/arXiv.2505.23735. arXiv: 2505.23735 [cs]. (Visited on 09/10/2025).

[DG24] Tri Dao and Albert Gu. "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality". In: *arXiv preprint arXiv:2405.21060* (2024).

[Hua+22] Weizhe Hua et al. "Transformer quality in linear time". In: *International conference on machine learning*. PMLR. 2022, pp. 9099–9117.

References II

[Lin+19] Juan Linde-Domingo et al. "Evidence That Neural Information Flow Is Reversed between Object Perception and Object Reconstruction from Memory". In: *Nature Communications* 10 (2019), p. 179. DOI: 10.1038/s41467-018-08080-2. URL: https://www.nature.com/articles/s41467-018-08080-2.

[Ope25] OpenAI. *GPT-OSS-120B & GPT-OSS-20B Model Card*. https://openai.com. Accessed: 2025-11-22. 2025.

[Sch12] Daniel L. Schacter. "Constructive Memory: Past and Future". In: *Dialogues in Clinical Neuroscience* 14.1 (2012), pp. 7–18. URL: https://www.tandfonline.com/doi/full/10.31887/DCNS.2012.14.1/dschacter.

References III

[Sch92] Jürgen Schmidhuber. "Learning to control fast-weight memories: An alternative to dynamic recurrent networks". In: *Neural Computation* 4.1 (1992), pp. 131–139.

[SIS21] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. "Linear Transformers Are Secretly Fast Weight Programmers". In: *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 2021, pp. 9355–9366. (Visited on 10/26/2025).

[Sun+25] Yu Sun et al. *Learning to (Learn at Test Time): RNNs with Expressive Hidden States*. 2025. DOI: 10.48550/arXiv.2407.04620. arXiv: 2407.04620 [cs]. (Visited on 11/19/2025).

References IV

[Tea+25] Kimi Team et al. *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025. DOI: 10.48550/arXiv.2510.26692. arXiv: 2510.26692 [cs]. (Visited on 10/31/2025).

[Vas+23] Ashish Vaswani et al. *Attention Is All You Need*. 2023. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs]. (Visited on 10/30/2025).

[Yan+] Songlin Yang et al. "Parallelizing Linear Transformers with the Delta Rule over Sequence Length". In: ().

[YKH25] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2025. DOI: 10.48550/arXiv.2412.06464. arXiv: 2412.06464 [cs]. (Visited on 10/26/2025).